

Day 5: Audio and vision on microcontrollers

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

- ▶ A microphone and a camera give much more data than an accelerometer: 16 000 samples each second, and 27 648 values for one small image.
- ▶ The model cannot take this data directly. Features make an audio signal small, and a small image and a narrow network make a vision model small.
- ▶ A device that listens all the time makes many decisions each second. The post-processing decides how many of them are false activations.
- ▶ In the lab you put a keyword spotting model and an image classifier on the XIAOML Kit, and you measure their latency and their RAM.

After this day you can:

- ▶ Compute a spectrogram and MFCC features from an audio signal.
- ▶ Describe the keyword spotting pipeline and the cascade design.
- ▶ Explain why peak activation memory limits vision models on a microcontroller.
- ▶ Deploy a keyword spotting model and an image classifier on the XIAOML Kit.
- ▶ Tune the post-processing to reduce false activations.

Part 1 Audio features and keyword spotting

Exercise: match three spectrograms to three spoken words

Part 2 Vision under microcontroller constraints

Exercise: calculate the peak activation memory of a MobileNet at two resolutions

Part 3 Streaming inference and detection

Exercise: design the cascade for a voice-controlled camera

- ▶ A microphone measures the air pressure. The board reads it as a stream of samples.
- ▶ Speech has frequencies up to about 8 kHz. So the sampling rate for speech is 16 kHz (Day 2: two times the highest frequency).
- ▶ Each sample has 16 bits.

One second of	Values
Motion, 50 Hz, 3 axes	150
Audio, 16 kHz	16 000
Bytes of the audio	32 000

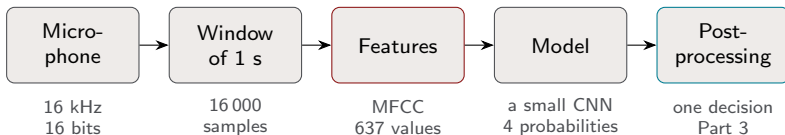
Key point: The microphone gives about 100 times as many values as the accelerometer of Day 3. A model for audio needs features more than a model for motion.

	Speech recognition	Keyword spotting
Task	Write each spoken word as text	Detect some words in a stream of sound
Vocabulary	Thousands of words	One to some tens of words
Model	Millions of parameters	Thousands of parameters
Runs on	A server or a phone	A microcontroller
Runs	After a request	All the time

- Uses: the wake word of a voice assistant, a voice command for a machine, a sound that starts a recording.
- The keyword spotter is the first stage. It wakes a larger system only when it hears the keyword. Part 3 gives this design.

Key point: Keyword spotting is the most frequent TinyML application: a small model that listens all the time with little power.

The pipeline of a keyword spotter



- ▶ The steps are the steps of Day 3: sense, pre-process, infer, post-process, act.
- ▶ The window moves: a new window starts each 0.5 s or each 0.25 s. The device makes some inferences each second.
- ▶ This part gives the features, the model, and the two types of errors. Part 3 gives the post-processing.

Why the model does not take the samples

The size

- ▶ One window has 16 000 values.
- ▶ A first layer with 20 neurons then has 320 000 weights.
- ▶ With the features, this layer has 12 740 weights.

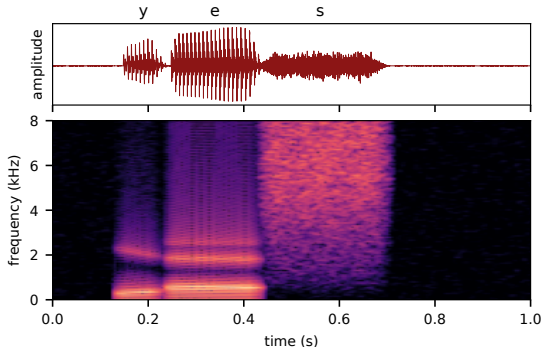
The content

- ▶ The same word gives different samples: a different voice, loudness, or start time.
- ▶ A sample alone has no meaning. The information is in the frequencies and in their change with time.

- ▶ Day 3 gave the same two reasons for motion data. For audio they are stronger, because the window is 50 times as large.

Key point: The features must show which frequencies are in the sound, and when. This picture is the spectrogram.

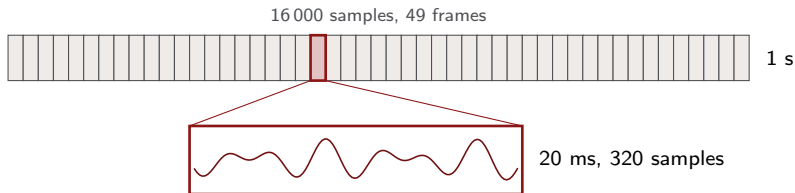
The spectrogram: frequency against time



- ▶ Top: the samples. You see when the sound is loud. Bottom: the spectrogram. Each column is the spectrum of a short piece of the signal. A bright point is a strong frequency.
- ▶ A vowel has horizontal bands at low frequencies. The sound “s” is noise above 4 kHz.

Source: A synthetic signal with the main properties of the word “yes”.

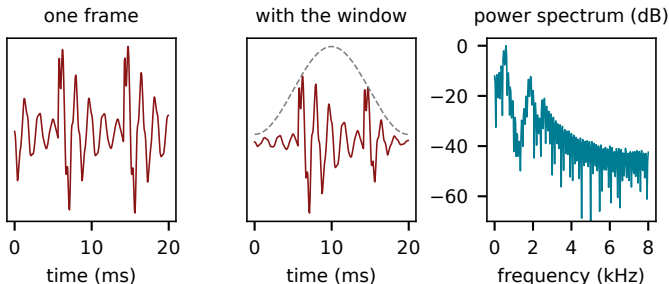
Step 1: cut the signal into frames



- ▶ A word changes all the time. In 20 to 40 ms, the sound is almost constant: the mouth does not move so fast.
- ▶ **Frame length:** the duration of one frame, here 20 ms.
- ▶ **Frame stride:** the time between the starts of two frames, here 20 ms. With a smaller stride, the frames overlap.
- ▶ Each frame gets one spectrum. The frames in order give the time axis of the spectrogram.

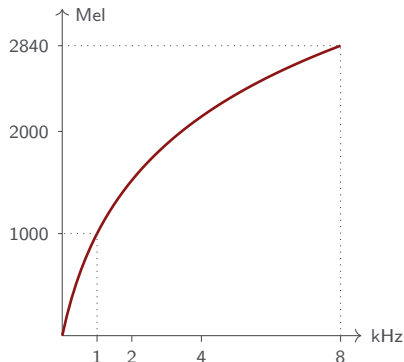
Source: "KWS Feature Engineering" of *Machine Learning Systems*.

Step 2: window function and FFT



- ▶ A frame starts and stops suddenly. The FFT sees the two jumps as frequencies that are not in the sound. The **Hamming window** makes the two ends of the frame small.
- ▶ The **FFT** gives the spectrum of the frame. With 512 points at 16 kHz, it has 257 bins with a width of 31.25 Hz, from 0 to 8 kHz.
- ▶ The features use the power of each bin: the square of the magnitude.

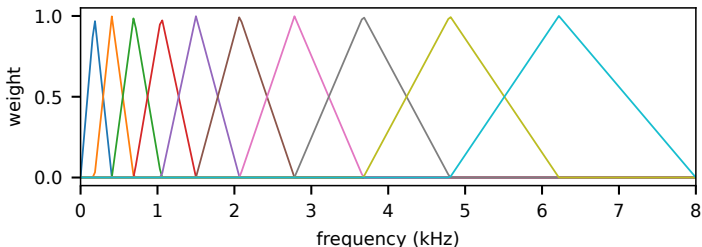
Step 3: the Mel scale



$$m = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right)$$

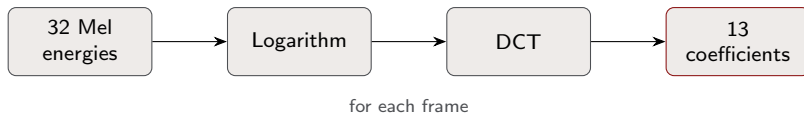
Frequency in Hz	Mel
500	607
1000	1000
2000	1521
4000	2146
8000	2840

- ▶ The ear separates two low tones better than two high tones. The Mel scale has equal steps for equal steps of the ear.
- ▶ The range from 0 to 1 kHz has 1000 Mel. The range from 4 to 8 kHz has only 694 Mel.



- ▶ Each triangle is one filter. It adds the power of the bins below it, with the weight of the triangle.
- ▶ The centres of the filters have equal distances on the Mel scale. So the filters are narrow at low frequencies and wide at high frequencies.
- ▶ One frame goes from 257 power values to one energy for each filter. A typical bank has 20 to 40 filters.
- ▶ The result for all frames is the **Mel spectrogram**. Other name: Mel filter bank energies (MFE).

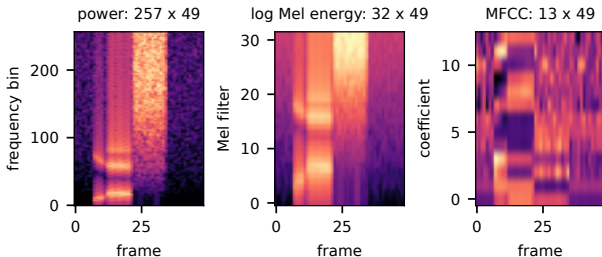
Step 4: logarithm and DCT



- ▶ **Logarithm.** The ear hears loudness on a logarithmic scale. The logarithm also makes a loud voice and a quiet voice more equal.
- ▶ **Discrete cosine transform (DCT).** The energies of neighbour filters are almost equal. The DCT writes the 32 values as a sum of cosine curves.
- ▶ The first coefficients give the smooth form of the spectrum. This form separates the speech sounds. The last coefficients give the fine detail.
- ▶ Keep the first 12 or 13 coefficients. These are the **Mel-frequency cepstral coefficients (MFCC)**.

Source: kit lab "KWS Feature Engineering" of *Machine Learning Systems*.

From 16 000 samples to 637 values



After the step	Size	Values
Samples of 1 s	16 000	16 000
Frames and FFT	49 frames of 257 bins	12 593
Mel filter bank	49 frames of 32 energies	1568
DCT	49 frames of 13 coefficients	637

The model gets 25 times fewer values than the microphone gives.

Source: The steps for the synthetic word "yes". The picture of the MFCC scales each coefficient alone, so that all rows are visible.

	Spectrogram	Mel energies (MFE)	MFCC
Steps	Frames, FFT	and the Mel filters, the logarithm	and the DCT
Values for 1 s	Most	Fewer	Fewest
Keeps	All detail of the frequencies	The detail that the ear hears	The smooth form of the spectrum
Good for	Machines, music, sounds of animals	Sounds of the environment	Speech: keywords, speakers

- ▶ Edge Impulse Studio gives the three as processing blocks. You select one in the lab.
- ▶ The feature block is a design decision. Train the model with two blocks, and compare the accuracy and the time on the device.

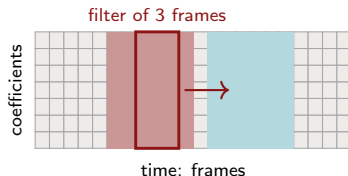
Source: "KWS Feature Engineering" of *Machine Learning Systems*.

The model: a small CNN on the MFCC image

Layer	Output	Parameters
Input: 49 frames of 13 coefficients	49×13	none
Convolution, 8 filters of 3 frames, ReLU	49×8	320
Max pooling, 2 frames	25×8	none
Convolution, 16 filters of 3 frames, ReLU	25×16	400
Max pooling, 2 frames	13×16	none
Fully connected, softmax	4	836
Total		1556

- ▶ The convolutions go along the time axis. The 13 coefficients of a frame are the channels of the first layer.
- ▶ In `int8`, the weights need about 1.6 KB of flash.

Source: "Keyword Spotting (KWS)". Parameter counts: the training notebook of the repository XIAO-ESP32S3-Sense.



- ▶ A keyword can start at any time in the window.
 - ▶ A filter looks at 3 frames: 60 ms. It moves along the window with the same weights.
 - ▶ So the filter finds a sound at each position. A fully connected layer must learn each position alone.
 - ▶ The pooling layers make the result less sensitive to a small shift.
- ▶ Larger keyword models use depthwise separable convolutions on the image of time and frequency. Day 6 explains this layer.

Source: Model types for keyword spotting: Zhang et al., "Hello Edge: Keyword Spotting on Microcontrollers", 2017.

Four classes: two keywords, unknown, and noise

Class	Content
YES	The first keyword
NO	The second keyword
UNKNOWN	Other words
NOISE	No speech: the room, a fan, a street

- ▶ A softmax always gives probabilities with the sum 1.
- ▶ With two classes only, each sound is “yes” or “no”: a door, a cough, the word “yesterday”.
- ▶ The two other classes give the model a correct answer for a sound that is not a keyword.

- ▶ The device hears a keyword only some seconds each day. Almost all windows are noise or other words.

Key point: The classes UNKNOWN and NOISE are the classes that the device sees most of the time. Their quality decides the false activations.

Source: Classes: kit lab “Keyword Spotting (KWS)” of *Machine Learning Systems*.

A public dataset

- ▶ Speech Commands: 35 words, more than 1000 clips of each word, 1 s for each clip, many speakers.
- ▶ It gives the keywords and the class UNKNOWN.
- ▶ The rules of Day 2 apply: all speakers of the test set are new speakers, and each class has a similar number of clips.
- ▶ Data augmentation makes more clips from one clip: add noise, move the word in the window, hide some frames or some coefficients.

Your recordings

- ▶ The keywords with your voice and your microphone.
- ▶ The noise of your room.
- ▶ Cut each recording into clips of 1 s.

Source: Dataset: Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition", 2018; kit lab "Keyword Spotting (KWS)".

Two errors: false accept and false reject

	Device says keyword	Device says nothing
Keyword spoken	Correct	False reject
No keyword	False accept	Correct

► **False reject rate (FRR):**
the part of the spoken keywords that the device misses.

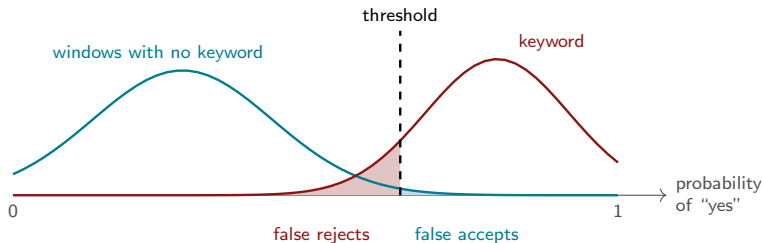
► **False accept rate (FAR):**
the part of the other windows that start the device.

- The user feels the two errors in different ways. A false reject: “I must say it again.” A false accept: “It starts alone.”
- One accuracy number mixes the two errors. For a keyword spotter, report the two rates.

Key point: For an always-on device, report the false accepts for each hour, and the false reject rate.

Source: Metrics: *Machine Learning Systems*, Volume I, chapter 4.

The threshold moves the two errors



- ▶ The device accepts a keyword when its probability is above a threshold.
- ▶ A higher threshold gives fewer false accepts and more false rejects. A lower threshold does the opposite.
- ▶ The model fixes the two curves. The threshold selects the point on the trade-off. You can change it after the training.

An always-on device makes many decisions

Quantity	Value
New window	each 0.5 s
Inferences in one hour	7200
Inferences in one day	172 800
False accept rate of 0.1 percent: false accepts in one hour	7.2
The same rate: false accepts in one day	173
Rate for one false accept in one day	0.0006 percent

- ▶ A model with 99.9 percent of correct windows starts the device 7 times each hour with no reason.
- ▶ The model alone cannot reach 0.0006 percent. The post-processing of Part 3 uses more than one window for one decision.

Key point: A very small error rate for each window becomes a large number of errors, because the device never stops.

Source: The values are a calculation for one window each 0.5 s.

The features

- ▶ 49 FFTs, the filter bank, and the DCT for each window.
- ▶ The Studio estimates 675 ms and 16 KB of RAM for the MFCC block of the kit lab, on a board with an ESP32.
- ▶ The features can need more time than the model. This is the result of Day 3 for motion data also.
- ▶ To make the device faster, change the features first: a shorter FFT, fewer coefficients, a larger frame stride.
- ▶ In the lab you read the two times from the Serial Monitor: the time for the features (DSP) and the time for the classification.

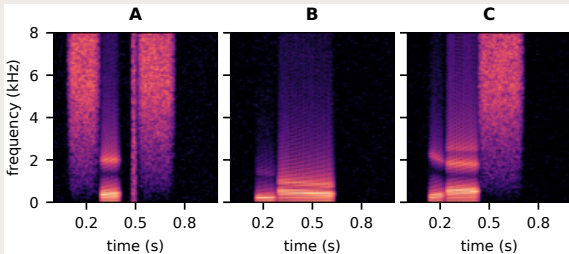
The model

- ▶ 1556 parameters.
- ▶ In `int8`, the largest sum of an input and an output is $637 + 392 = 1029$ bytes.

Source: Estimates of Edge Impulse Studio for the ESP-EYE: kit lab “Keyword Spotting (KWS)”.

Exercise: match spectrograms to words

The three spectrograms show the words “yes”, “no”, and “six”. Each picture has 1 s.



1. Which word is A, which is B, and which is C? Give one reason for each picture.
2. Which two words are most difficult to separate for a model? Why?
3. A model sees only the frequencies below 2 kHz. Which two words can it still separate?

Answer: match spectrograms to words

1. **A is “six”**. It has noise above 4 kHz at the start and at the end: the two sounds “s”. Between them is a short vowel, then a pause, and a short burst: the sound “k”.
B is “no”. It has no noise at high frequencies. All its power is in bands below 1.5 kHz: the sound “n” and a vowel.
C is “yes”. It starts with bands at low frequencies, and one band moves down: the sounds “y” and “e”. It ends with noise above 4 kHz: the sound “s”.
2. “yes” and “six”. The two words have a vowel and then the sound “s”. Only the start is different.
3. “no” against the two other words: the vowel of “no” has its bands below 1 kHz, and the vowels of “yes” and “six” have a band near 2 kHz. Below 2 kHz, the sound “s” has almost no power, so “yes” and “six” look more equal.

A model does this work with the MFCC image. It looks for the same patterns: where is the power, and when does it change.

Part 1 Audio features and keyword spotting

Exercise: match three spectrograms to three spoken words

Part 2 Vision under microcontroller constraints

Exercise: calculate the peak activation memory of a MobileNet at two resolutions

Part 3 Streaming inference and detection

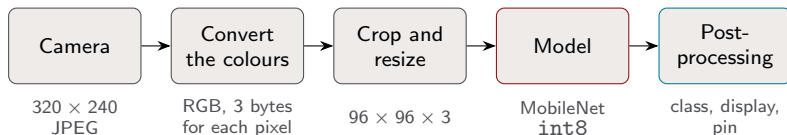
Exercise: design the cascade for a voice-controlled camera

Input of a model	Values
Motion features (Day 3)	63
MFCC of 1 s of audio	637
Image 96×96 , RGB	27 648
Image 160×160 , RGB	76 800
Image 224×224 , RGB	150 528

- ▶ An image has no feature step. The model takes the pixels.
- ▶ The first layers of the CNN do the work of the features. So they hold large tensors.
- ▶ A camera gives much more than the model takes: 320×240 pixels or more.

Key point: For motion and audio, the features make the input small. For an image, only a smaller image makes the input small.

The pipeline of an image classifier



- ▶ The steps before the model are the pre-processing. They need time and RAM, as the features of the audio pipeline do.
- ▶ The model is a small CNN from the MobileNet family.
- ▶ This part gives the limits of the model first, then the two controls that make it fit, then the training, then the pre-processing.

Source: Pipeline of the sketch: kit lab "Image Classification" of *Machine Learning Systems*.

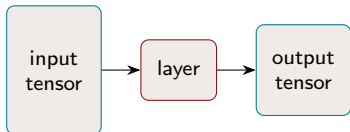
Three budgets, three properties of the model

Budget	Property of the model	Depends on	XIAO ESP32S3
Flash	The number of parameters	The width of the layers	About 3.3 MB for the program
RAM	The peak activation memory	The image size and the width	About 300 KB free, and 8 MB of PSRAM
Time	The number of operations (MACs)	The image size and the width	Your target for the frame rate

- Day 1 gave the three counts. A CNN has few parameters for its size, because the filters are small. But each filter makes a large feature map.

Key point: The motion model and the audio model of this course are far below the three budgets. For a vision model, the RAM fails first.

Source: Flash and RAM of the XIAO ESP32S3: the build output of the Day 1 lab.

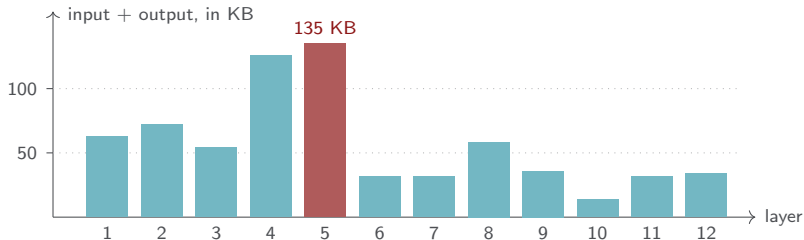


the two tensors exist at the same time

- ▶ When a layer runs, its input and its output are in the RAM.
 - ▶ After the layer, the runtime can use the input buffer again.
 - ▶ The peak is the largest sum of an input and an output, for all layers.
 - ▶ A skip connection keeps one more tensor.
- ▶ In `int8`, one value is one byte. So the peak in values is the peak in bytes.
 - ▶ The arena also holds the data of the interpreter. For a vision model, this part is small against the activations.

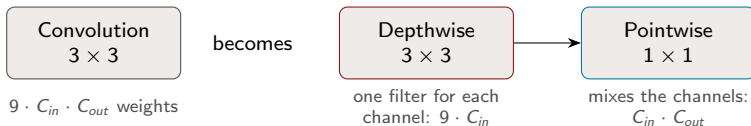
Key point: The size of the model file says nothing about the peak. You must look at the feature maps.

Where the peak is: the first layers



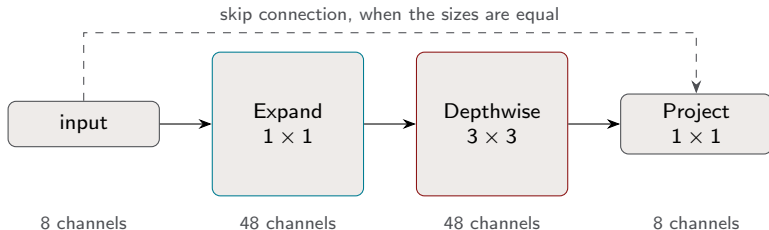
- ▶ The bars show MobileNetV2 with a width multiplier of 0.35 and an image of 96×96 , in int8.
- ▶ The first layers have feature maps of 48×48 . The peak is at layer 5, with 48 channels.
- ▶ Each stride of 2 makes the map four times smaller. The other 50 layers stay below 30 KB.

Source: A calculation with the Keras model of MobileNetV2.



- ▶ A normal convolution filters the space and mixes the channels in one step.
- ▶ MobileNet does the two jobs in two layers. Example with 48 input channels and 48 output channels: 20 736 weights become $432 + 2304 = 2736$ weights.
- ▶ The number of operations decreases by the same factor: about 8.
- ▶ The feature maps do not become smaller. The layer saves flash and time, but it saves no RAM.

Key point: MobileNet solves the flash budget and the time budget. The RAM budget needs two more controls.



- ▶ The block has a narrow input and a narrow output. Inside, the expand layer makes 6 times as many channels.
- ▶ The depthwise layer works on the wide tensor. This is the place of the peak.
- ▶ The narrow tensors between the blocks keep the skip connection small.

Source: The channel numbers are the numbers of the second block for a width multiplier of 0.35.
Block: Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks", 2018.

Image	Parameters	MACs	Peak in int8	Against 96
96×96	410 208	10.7 million	135 KB	1.0
160×160	410 208	29.6 million	375 KB	2.8
224×224	410 208	58.0 million	735 KB	5.4

- ▶ The table shows MobileNetV2 with a width multiplier of 0.35, with no classification layer.
- ▶ The filters do not change with the image size. So the number of parameters and the flash use stay equal.
- ▶ Each feature map grows with the area of the image. The peak and the operations grow with the square of the side: $(160/96)^2 = 2.8$.

Key point: The resolution is the strongest control for the RAM and for the time. It costs accuracy when the object becomes too small in the image.

Source: A calculation with the Keras model of MobileNetV2.

Control 2: the width multiplier

Width multiplier	Parameters	MACs	Peak in int8
α			
0.35	410 208	10.7 million	135 KB
0.50	706 224	17.6 million	135 KB
1.00	2 257 984	55.0 million	270 KB

- ▶ The table shows MobileNetV2 for an image of 96×96 .
- ▶ α multiplies the number of channels of each layer. The result is rounded to a multiple of 8.
- ▶ A layer has $C_{in} \cdot C_{out}$ weights. So the parameters and the operations grow about with α^2 .
- ▶ A feature map grows with α . The values 0.35 and 0.50 give the same first layers after the rounding, so the peak is equal.

Key point: The width is the control for the flash. The resolution is the control for the RAM. The two change the time.

Source: A calculation with the Keras model of MobileNetV2.

Model in Edge Impulse Studio	Image	RAM	Flash
MobileNetV1, $\alpha = 0.10$	96×96	53 KB	101 KB
MobileNetV2, $\alpha = 0.35$	96×96	233 KB	546 KB
MobileNetV2, $\alpha = 1.0$	160×160	1.3 MB	2.6 MB

- ▶ The Studio gives MobileNetV1 and MobileNetV2 with α from 0.05 to 1.0, for images of 96×96 and 160×160 .
- ▶ The lab uses MobileNetV2 with $\alpha = 0.35$ and 96×96 : the second row.
- ▶ The RAM estimate of the Studio is larger than the peak of 135 KB. It holds the complete arena and the buffers of the library.

Key point: Select the smallest model that reaches your accuracy target. Start small, and go to a larger model only when the accuracy is too low.

Source: "Image Classification" of *Machine Learning Systems*.

- ▶ The question: is a person in the image, or not?
 - ▶ It is the vision task with the same role as keyword spotting: a small model that is always on and wakes a larger system.
 - ▶ The dataset comes from a large public dataset of photos. Each image has the label “person” or “not person”.
 - ▶ A task with two classes needs less from the model than a task with 1000 classes. So a small width and a small image are enough.
 - ▶ Uses: a light that follows the people in a room, a doorbell, a camera that records only with a person in the image.
- The target of the paper
- ▶ A microcontroller with about 250 KB of RAM.
 - ▶ Mobile models reach an accuracy of 85 to 90 percent in this limit.

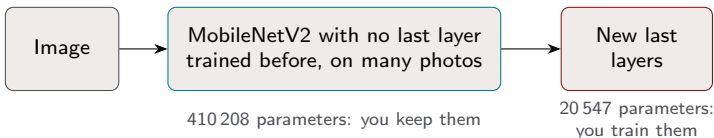
Source: Chowdhery et al., “Visual Wake Words Dataset”, 2019.

The PSRAM: more RAM, at a price

	Internal RAM	PSRAM
Size	512 KB, about 300 KB free	8 MB
Place	In the chip	A second chip in the module
Access	Direct	Through a serial bus: slower
Use it for	The arena of a small model	The image buffers, the arena of a large model

- ▶ One image of 320×240 with 3 bytes for each pixel needs 230 400 bytes. The camera needs the PSRAM for its frame buffer.
- ▶ A model with a peak above 300 KB runs only with the arena in the PSRAM. You measure the price in time in the lab.

Key point: The PSRAM removes the RAM limit of this board. A product with a smaller chip has no PSRAM.



- ▶ A model for images needs thousands of images for each class. You have about 50.
- ▶ The first layers of a CNN learn edges, colours, and textures. These features are good for all photos.
- ▶ So you take a model that learned them from a large dataset. You train only the last layers for your classes.

Key point: With transfer learning, 50 images for each class can be enough, and the training needs minutes.

Source: "Image Classification" of *Machine Learning Systems*.

Setting	Value
Base model	MobileNetV2, $\alpha = 0.35$, 96×96
Last layers	16 neurons, then one output for each class
Dropout	0.1
Epochs	20
Learning rate	0.0005

- ▶ **Data augmentation:** each epoch sees the images with small random changes: a flip, a crop, a change of the brightness.
- ▶ **Fine-tuning:** some more epochs with all layers and a very small learning rate. It is an option.

- ▶ The base model fixes the image size. A base model that was trained with 96×96 needs images of 96×96 .
- ▶ After the training, the Studio quantizes the model to `int8`, as on Day 4. The calibration set comes from your training images.

Source: Settings: kit lab “Image Classification” of *Machine Learning Systems*.

Capture: the camera gives a format

Format	One pixel	Bytes for 320×240	Note
Grayscale	1 byte: the brightness	76 800	No colour
RGB565	2 bytes: 5, 6, and 5 bits	153 600	Format of small displays
RGB888	3 bytes: 8 bits for each colour	230 400	Format of the model
JPEG	Compressed	Some thousands, not constant	The sketch must decode it

- ▶ The camera sensor of the kit gives these formats and many frame sizes. The model takes RGB888 with its own size.
- ▶ The sketch of the lab captures a JPEG image of 320×240 and decodes it to RGB888.

Source: Formats and frame size of the sketch: kit lab "Image Classification" of *Machine Learning Systems*.



- ▶ The camera image is wider than high. The model image is a square. In the resize step, each output pixel takes the value of the nearest input pixel, or a mean of its neighbours.
- ▶ **Crop:** cut the sides. The objects keep their form, and the sides of the scene are lost.
- ▶ **Squash:** resize the two axes with different factors. The complete scene stays, and the objects become narrow.

Key point: The object must still be visible in 96×96 pixels. Look at the small image before you train the model.

Step	In the training	On the device
Source of the image	A photo file, or the camera of the kit	The camera of the kit
Crop or squash	A setting of the Studio	Code of the sketch
Order of the colours	Red, green, blue	The same order
Range of the values	0 to 1, or -1 to 1	The same range, then the scale and the zero point of the input tensor

- ▶ A difference in one row gives a model that works on the laptop and fails on the board. No error message shows it.
- ▶ Day 3 gave this rule for the features. For images, the camera, the light, and the background are also a part of it.

Key point: Collect the images with the device that runs the model.

Model, int8	Image	Pre-process	Infer	Frames/s
MobileNetV2 of SenseCraft AI, three classes	224×224	110 ms	426 ms	1.8
MobileNetV2, $\alpha = 0.35$, of Edge Impulse Studio, three classes	96×96	81 ms	205 ms	3.4

- ▶ The numbers come from the kit lab. Its author measured them on a XIAO ESP32S3 Sense with the firmware of SenseCraft AI.
- ▶ A person classifier with MobileNetV2 and $\alpha = 0.35$ needs a peak current of 160 mA at 5.23 V: about 830 mW.
- ▶ The Studio estimated 1160 ms for the second model, for a board with an older processor. The estimate of a tool is not a measurement.
- ▶ Your sketch uses a different library: you measure your own times.

Source: Kit lab “Image Classification” of *Machine Learning Systems*.

What it gives

- ▶ One class for the complete image.
- ▶ A probability for each class.

What it does not give

- ▶ The position of the object.
- ▶ The number of objects.
- ▶ An answer “none of my classes”, if no class for it exists.

- ▶ Add a class “background” with images of the empty scene. It has the role of the classes UNKNOWN and NOISE of Part 1.
- ▶ One object must fill a large part of the image. Two objects of different classes in one image give an unclear result.
- ▶ For the position and the number of objects, you need a detection model. Part 3 gives one that fits a microcontroller.

Key point: A classifier answers one question for one image. Design the scene of the camera so that this question has a clear answer.

Exercise: the peak activation memory

The first layers of MobileNetV2 with $\alpha = 0.35$, for an image of 96×96 , in `int8`. No layer of the table has a skip connection.

Layer	Type	Input	Output
1	Convolution, stride 2	$96 \times 96 \times 3$	$48 \times 48 \times 16$
2	Depthwise	$48 \times 48 \times 16$	$48 \times 48 \times 16$
3	Project	$48 \times 48 \times 16$	$48 \times 48 \times 8$
4	Expand	$48 \times 48 \times 8$	$48 \times 48 \times 48$
5	Depthwise, stride 2	$48 \times 48 \times 48$	$24 \times 24 \times 48$

1. Calculate the sum of the input and the output for each layer. Which layer has the peak? Give the peak in KB.
2. The image is now 160×160 . Calculate the new peak with the factor between the two image areas.
3. The board has about 300 KB of free internal RAM. Which model runs with no PSRAM?
4. The frame buffer of the camera needs 230 400 bytes. Where do you put it?

1. The sums in bytes:

Layer	Input	Output	Sum
1	27 648	36 864	64 512
2	36 864	36 864	73 728
3	36 864	18 432	55 296
4	18 432	110 592	129 024
5	110 592	27 648	138 240

The peak is at layer 5: 138 240 bytes, or 135 KB.

2. The factor is $(160/96)^2 = 2.78$. The new peak is $138\,240 \cdot 2.78 = 384\,000$ bytes, or 375 KB.
3. Only the model for 96×96 . Its peak of 135 KB is below 300 KB. The peak of 375 KB is above it.
4. In the PSRAM. With the frame buffer of 225 KB in the internal RAM, the model for 96×96 has no space also: $225 + 135 = 360$ KB.

Part 1 Audio features and keyword spotting

Exercise: match three spectrograms to three spoken words

Part 2 Vision under microcontroller constraints

Exercise: calculate the peak activation memory of a MobileNet at two resolutions

Part 3 Streaming inference and detection

Exercise: design the cascade for a voice-controlled camera

In the training

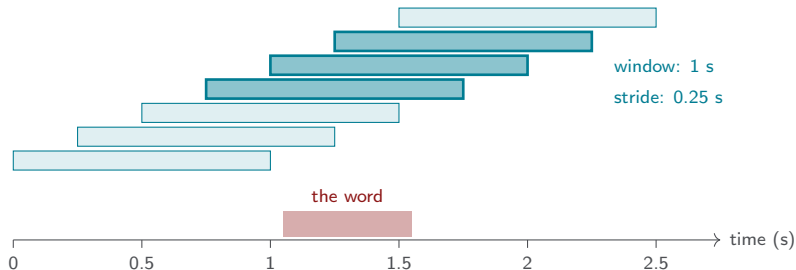
- ▶ One clip of 1 s, with the word in the centre.
- ▶ One clip, one label, one result.

On the device

- ▶ A stream of sound with no start and no end.
 - ▶ Nobody says where a word starts.
 - ▶ The product needs one event for each spoken keyword.
-
- ▶ The model is the same. The code around the model must cut the stream into windows and make events from many results.
 - ▶ This code is the post-processing. It has no weights, but it has parameters that you must tune.

Key point: A good accuracy on clips does not give a good device. The device is the model and the post-processing together.

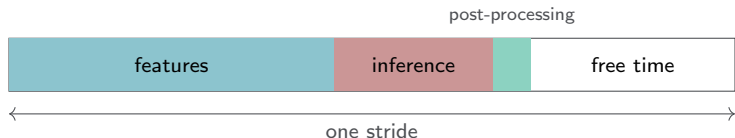
The sliding window



- ▶ A new window starts after each stride. The windows overlap.
- ▶ A small stride gives more windows that hold the complete word: three in the figure. It also gives more inferences each second.
- ▶ A stride equal to the window gives the fewest inferences. But a word at the limit of two windows is cut into two parts.

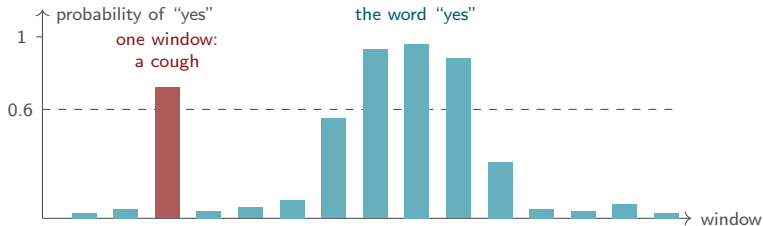
Key point: The stride decides the number of inferences for each second, and the chance that one window holds the complete word.

The time budget of a stream



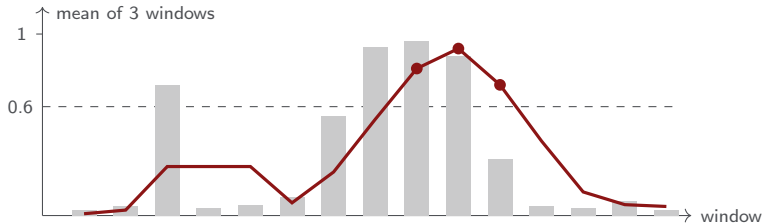
- ▶ The work for one window must end before the next window starts.
- ▶ Rule: the time for the features, plus the time for the inference, plus the time for the post-processing is smaller than the stride.
- ▶ If the rule fails, the device loses samples, or the results come later and later. Day 3 counted these late samples.
- ▶ The microphone must still record when the model runs. The sketch reads the samples in a second task, or with an interrupt.

Key point: Measure the two times on the board first. Then select the stride.

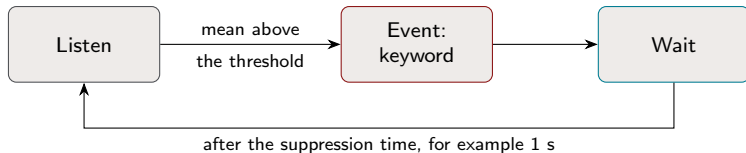


- ▶ The model gives one probability for each window. A real word gives a high value in some windows in sequence.
- ▶ A short noise gives a high value in one window.
- ▶ With a threshold of 0.6 on each window, this stream gives four events: one false accept, and three events for one word.

Smoothing: the mean of the last windows



- ▶ The post-processing keeps the last N results and calculates their mean. Here $N = 3$.
- ▶ The cough gives a mean of 0.27: below the threshold. The false accept disappears.
- ▶ The word still passes the threshold, one window later than before. Smoothing costs a delay.



- ▶ The smoothed value stays above the threshold for some windows. Without one more rule, one word gives some events.
- ▶ After an event, the device accepts no new event for a suppression time.
- ▶ Three parameters of the post-processing: the number of windows N , the threshold, and the suppression time.

Key point: The post-processing is a small state machine. You can test it on the laptop with recorded probabilities, with no board.

	Small N , low threshold	Large N , high threshold
False accepts	More	Fewer
False rejects	Fewer	More: a short word does not fill N windows
Delay of the event	Short	Longer: about N strides

- ▶ Part 1: a false accept rate of 0.1 percent for each window gives 7.2 false accepts each hour.
- ▶ If three windows were independent, an event with three windows would have a rate of 0.001^3 : one false accept in 16 years.
- ▶ The windows overlap, so their errors are not independent. The real gain is much smaller. Measure it.

Key point: Smoothing moves the device along the same trade-off as the threshold of Part 1, and it adds a delay.

Tune the post-processing with a test

Test	What you do	What you count
False accepts	Normal speech with no keyword, for a fixed time	The events
False rejects	Say the keyword 20 times, with pauses	The missed keywords
Delay	Say the keyword and look at the display	The time to the event

- ▶ Change one parameter, and repeat the three tests. Write each result in a table.
- ▶ Test with a second speaker, and with noise in the room. The tuning for your voice alone is too good.
- ▶ The check of the lab: no false activation in 30 seconds of normal speech.

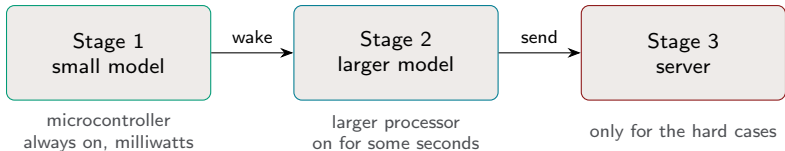
Key point: The tuning is a measurement on the device, not a setting that you take from a different project.

- ▶ A camera gives some frames each second. Each frame gives a class.
- ▶ One frame can be wrong: a hand in front of the camera, a reflection, a blurred image.
- ▶ Accept a class only when the last N frames agree, or when the mean probability is above a threshold.

Frame	Class
1	background
2	background
3	wheel
4	background
5	box
6	box
7	box

- ▶ With $N = 3$, the device reports “box” at frame 7. It never reports “wheel”.
- ▶ At 3 frames each second, the answer comes one second after the object. For a slow scene, this delay is no problem.

The cascade: a small model wakes a large model



- ▶ Stage 1 answers an easy question all the time: “Is this the keyword?” or “Is a person there?”
- ▶ Stage 2 answers the difficult question, but only after stage 1 wakes it. Most of the time it is off.
- ▶ A voice assistant works in this way: a keyword spotter in the device, and the recognition of the complete sentence in the cloud.

Source: Design of voice assistants: kit lab “Keyword Spotting (KWS)” of *Machine Learning Systems*.

Why a cascade saves energy

$$P_{\text{mean}} = P_1 + f \cdot P_2 \quad f = \frac{\text{time with stage 2 on}}{\text{total time}}$$

Quantity	Value
Stage 1, always on: P_1 (example value)	10 mW
Stage 2, vision model on the kit: P_2	830 mW
Events each day, with 5 s of stage 2 for each event	22
Part of the time with stage 2 on: f	0.13 percent
Mean power with the cascade	11.1 mW
Mean power with stage 2 always on	830 mW

The cascade needs 75 times less power. Stage 2 is almost always off, so the power of stage 1 decides the result.

	Stage 1	Stage 2
Runs	All the time	After a wake signal
Must be	Small and low in power	Accurate
Its worst error	A false reject: stage 2 never sees the event	A false accept: the product does a wrong action
Tune it for	Few false rejects. Some false accepts are permitted.	Few false accepts

- ▶ Stage 2 can check stage 1 again. So stage 1 can use a lower threshold than a device with one stage.
- ▶ The delay is the sum: the smoothing of stage 1, the time to wake stage 2, and the inference of stage 2.

Key point: Each stage removes most of the cases, so that the next stage works rarely.

	Classification	Object detection
Question	What is in the image?	What is where, and how many?
Output	One probability for each class	For each object: a class, a score, and a box with four numbers
Number of outputs	Fixed: the number of classes	Not fixed: it depends on the image
Typical models	MobileNet	SSD, YOLO

- ▶ A detector looks at many positions and many sizes. For each of them, it predicts a class and a box.
- ▶ Then a step with no weights removes the boxes that show the same object. Day 8 gives the details.

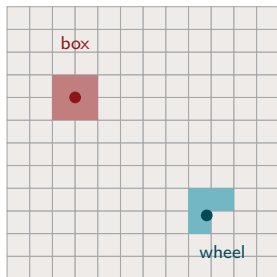
Key point: Detection is classification at many places of the image, plus the geometry of the box. It costs much more than one classification.

Why SSD and YOLO do not fit

Property	A detector for a phone or a Raspberry Pi	The microcontroller
Image size	320×320 or more: small objects must stay visible	96×96 fits
Peak activation memory	1.5 MB to 2.9 MB for the first layers alone	About 300 KB of internal RAM
Model file	Several MB	3.3 MB for the complete program
Outputs	Thousands of boxes, then a filter step	Little time and little RAM for it

- ▶ The peak values are a calculation for MobileNetV2 at 320×320 in `int8`, for the widths 0.35 and 1.0.
- ▶ The image size is the main problem. In a small image, a small object has too few pixels.

Source: Model sizes of the detectors: kit lab “Object Detection” of *Machine Learning Systems*.

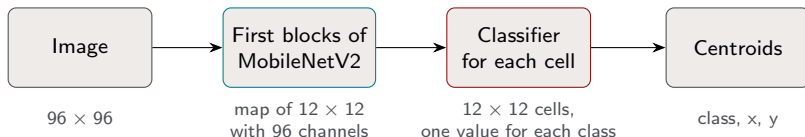


12 × 12 cells for 96 × 96 pixels

- ▶ FOMO: “Faster Objects, More Objects”, a method of Edge Impulse.
- ▶ It divides the image into cells of 8×8 pixels.
- ▶ For each cell, it gives a probability for each class and for the background.
- ▶ Neighbour cells of one class make one object. The output is the centre of the object: its centroid.
- ▶ It gives no box and no size.

Key point: Many products need only the class, the position, and the number of objects. FOMO gives these three, at the cost of a classifier.

Source: Kit lab “Object Detection” of *Machine Learning Systems*.



- ▶ A normal MobileNet makes the map smaller in five steps, down to 3×3 . FOMO stops after three steps: one cell for 8×8 pixels.
- ▶ The last layers are convolutions of 1×1 . They classify each cell with the same weights.
- ▶ The cut model is small: the first blocks have about 17 600 parameters for a width multiplier of 0.35.
- ▶ Training: you draw boxes. The tool makes the label of each cell from the centre of each box.

Source: Method: kit lab “Object Detection”. Parameter count: a calculation with the Keras model of MobileNetV2, cut at the map of 12×12 .

Number of the kit lab	Value
RAM, estimate of the Studio	250 KB
Flash, estimate of the Studio	80 KB
Latency on the XIAO ESP32S3	143 ms
Frames each second	about 7

The limits

- ▶ No size of the object.
- ▶ Two objects in neighbour cells become one object.
- ▶ The objects must have a similar size, near the size of a cell or some cells.

- ▶ The metric is the F1 score, not the accuracy: almost all cells are background, so a model that finds nothing has a high accuracy.
- ▶ The method shows a general idea: change the question until the model fits.

Source: Kit lab "Object Detection" of *Machine Learning Systems*. Its author measured the latency.

Which model for which question

The product must know	Model	Smallest device
Is the object there?	Image classifier	Microcontroller
Where are the objects, and how many?	FOMO	Microcontroller
The size of each object, or objects of very different sizes	SSD, YOLO	Raspberry Pi (Day 8)
A description of the scene in words	Vision-language model	Raspberry Pi with 8 GB (Day 10)

- ▶ The table has the form of a cascade also: a classifier on a microcontroller can wake a detector on a Raspberry Pi.
- ▶ Week 3 connects the two boards of this course in this way.

Key point: Ask what the product must know. The answer selects the smallest model, and the smallest model selects the device.

Exercise: design a cascade

A camera for a garden has a battery of 7.4 Wh. The user says “photo”. The camera takes an image, and it stores the image only if a bird is in it.

Keyword model, always on (example value)	10 mW
Camera and image model on the XIAO ESP32S3	830 mW
Time of stage 2 for one event	5 s
Real commands each day	20
False accepts of the keyword stage each day	2

1. Draw the cascade. Which stage is always on? What is the wake signal?
2. Calculate the mean power and the battery life. Compare with a camera that runs the image model all the time.
3. The user says: “I must say the word two times.” Which parameter do you change? What does the change cost?
4. What does the image model do with a false accept of stage 1?

1. Stage 1: the microphone and the keyword model, always on. The wake signal is the event “photo” after the smoothing. Stage 2: the camera and the image classifier with the classes “bird” and “background”. Then the device stores the image.
2. Stage 2 is on for $22 \times 5 = 110$ s each day: $f = 110/86\,400 = 0.13$ percent. The mean power is $10 + 0.0013 \times 830 = 11.1$ mW. The battery gives $7.4/0.0111 = 669$ hours: 28 days. With stage 2 always on, the battery gives $7.4/0.83 = 8.9$ hours.
3. A false reject. Use a lower threshold, or fewer windows for the mean. The cost: more false accepts. Each of them costs 5 s of stage 2, which is little energy. So stage 1 can be tolerant.
4. It takes an image and finds no bird. It stores nothing. Stage 2 is the second check of stage 1: the user sees no wrong photo.

The power of stage 1 decides the battery life. With 5 mW in place of 10 mW, the battery gives about 51 days.

- ▶ An audio model does not take samples. It takes a small image of the sound: MFCC features of short frames.
- ▶ A keyword spotting model needs a class for unknown words and a class for noise. You tune it with the false accept rate and the false reject rate.
- ▶ For a vision model, the peak activation memory decides if the model fits. A smaller image and a narrower network decrease it.
- ▶ A stream gives many results each second. Smoothing and a threshold turn them into one decision. A cascade keeps the large device asleep.

Key point: On a microcontroller, the model is one part of a pipeline. The features before the model and the post-processing after the model decide the result as much as the model.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 6 “Network Architectures”, section “CNNs: Spatial Pattern Processing”.
- ▶ *Machine Learning Systems*, XIAOML Kit labs: “Keyword Spotting (KWS)”, “KWS Feature Engineering”, and “Image Classification”.

Papers

- ▶ “Hello Edge: Keyword Spotting on Microcontrollers”, Zhang et al., 2017.
- ▶ “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition”, Warden, 2018.
- ▶ “Visual Wake Words Dataset”, Chowdhery et al., 2019.
- ▶ “MCUNet: Tiny Deep Learning on IoT Devices”, Lin et al., 2020.

Tools

- ▶ Edge Impulse documentation, with the pages on MFCC and on FOMO:
<https://docs.edgeimpulse.com>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai. github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook, GPL-3.0.